

DIYNEZA.

Complete Build Specification

A full product & engineering brief for building DIYNEZA — an all-in-one restaurant management platform (POS, inventory, KDS, QR ordering & multi-outlet operations). Hand this document to an AI coding agent to build the system module by module.

Master prompt + feature spec + data model + roadmap

Product: DIYNEZA • Founders: Meghansh Agarwal (CEO), Roop Jaiswal (Co-Founder) • diyneza.com

How to use this document: Copy the “Master Build Prompt” (Section 1) into your AI coding agent first. Then build module-by-module using Sections 4–6, pasting each module’s spec as you go. Sections 7–10 (data model, APIs, non-functional, roadmap) keep the build consistent. The public marketing website already exists (Next.js) — this brief is for the **product application** behind login.

1 • Master Build Prompt (paste this first)

ROLE

You are a senior full-stack engineering team. Build DIYNEZA, a production-grade, multi-tenant SaaS for restaurant management. Ship clean, typed, tested code.

PRODUCT

DIYNEZA is an all-in-one restaurant operating system. One platform for: POS & billing, menu, recipe inventory & costing, Kitchen Display System (KDS), QR/tablesides & online ordering, CRM & loyalty, multi-outlet management, analytics, staff/roles, and payments. Offline-first billing is mandatory.

STACK

- Frontend: Next.js (App Router) + React + TypeScript + Tailwind
- Mobile/POS terminal: responsive PWA (installable, offline via IndexedDB + sync)
- Backend: Supabase (Postgres + Auth + RLS + Realtime + Storage) OR Node/Nest API
- Payments: Stripe + Razorpay (pluggable provider interface)
- Messaging: Twilio (SMS/WhatsApp) + email (Resend/SES)
- Realtime: Supabase Realtime / WebSockets for KDS + live orders

ARCHITECTURE RULES

- Multi-tenant: every row scoped by `organization_id` + `outlet_id`; enforce with RLS.
- Role-based access control (RBAC): owner, manager, cashier, kitchen, waiter.
- Offline-first POS: queue transactions locally, sync on reconnect, conflict-safe.
- Everything API-first; the web app and POS terminal consume the same API.
- Audit log on all money/stock mutations.

DELIVERABLES

Build the modules in Section 4 in the phase order of Section 9 (MVP first). For each module: data model, API endpoints, UI screens, validations, tests. Start by scaffolding auth + multi-tenant + RBAC, then POS, then the rest.

2 · Product overview

What we're building and for whom.

- **Vision:** replace 5–7 disconnected restaurant tools with one fast, reliable, cloud platform.
- **Users:** restaurant owners, outlet managers, cashiers, kitchen staff, waiters; plus diners (QR ordering, no login).
- **Segments:** QSR & chains, cafes & bakeries, fine dining, cloud kitchens.
- **Business model:** SaaS subscription (Starter / Growth / Enterprise), 45-day free trial, 0% commission on direct orders.

3 · User roles & permissions (RBAC)

Role	Can do
Owner	Everything across all outlets: billing config, pricing, staff, reports, settings.
Manager	Single/assigned outlet: menu, inventory, staff shifts, discounts, outlet reports.
Cashier	POS billing, take orders, accept payments, basic refunds (with approval).
Kitchen	KDS only: view tickets, mark items prepared/ready.
Waiter	Take table orders on handheld, send to kitchen, request bill.
Diner (no login)	Scan QR, browse menu, order, pay, split bill.

4 · Modules & features (the core build)

1 Authentication & multi-tenant MVP

- Email/password + magic link auth; org sign-up creates an **organization** + first **outlet**.
- Invite staff by email with a role; per-outlet assignment.
- Row-level security: all data scoped to organization_id (+ outlet_id where relevant).
- Subscription/plan state on the organization (trial, active, plan tier, limits).

2 POS & Billing (offline-first) MVP

- Fast order screen: category → item → modifiers → cart; quick-keys layout.
- Order types: dine-in (table), takeaway, delivery, QR.
- Split bill, merge tables, discounts (% / flat / coupon), taxes, service charge, tips.
- Payments: cash, card, UPI, wallet, split-tender; print/share receipt.
- **Offline-first**: bills work without internet (IndexedDB queue), auto-sync & reconcile on reconnect.
- Shift open/close, cash drawer reconciliation, X/Z reports.
- Refunds & voids with reason + approval + audit trail.

3 Menu management MVP

- Categories, items, variants (size), modifiers/add-ons, combos.
- Per-outlet pricing & availability; happy-hour / time-based pricing.
- Item images, tags (veg/non-veg/spicy), prep station mapping, tax class.

4 Inventory & recipe costing V1

- Raw ingredients with units; **recipe cards** linking menu items → ingredient quantities.
- Auto stock deduction (to the gram) when a bill settles.
- Min/max thresholds, low-stock alerts, auto **Purchase Orders** to suppliers.
- Supplier directory, goods-received entry, wastage logging with reasons.
- Live food-cost % and ingredient-level margin reporting.

5 Kitchen Display System (KDS) MVP

- Realtime tickets from POS/QR routed by **prep station** (hot line, bar, dessert).
- Color-coded timers, bump/recall, item-level done, all-day counts.
- Server/waiter notification when a dish is ready.

6 QR / tableside & online ordering V1

- Per-table QR → diner browses menu, orders, pays — no app, no login.
- Branded online storefront for direct delivery/pickup (0% commission).
- Split & pay at table; orders flow straight to POS + KDS.
- Interactive table/floor map & seating status.

7 CRM & loyalty V2

- Auto-capture customers from orders; profiles, order history, spend.
- Loyalty points, tiers, coupons, prepaid/gift cards.
- Campaigns via SMS/WhatsApp/email (Twilio); win-back & birthday offers.

8 Multi-outlet / franchise V1

- Central menu & price management with per-outlet overrides.
- Consolidated cross-outlet reporting; recipe versioning; outlet-level roles.
- Inter-outlet stock transfer.

9 Analytics & reporting V1

- Dashboards: sales, top items, hourly traffic, payment mix, taxes.
- Inventory & wastage reports, food-cost, staff/shift performance.
- Exports (CSV/PDF); scheduled email reports.

10 Payments & integrations V1

- Pluggable payment provider interface: Stripe, Razorpay (UPI/cards/wallets).
- Aggregator order ingestion (Zomato/Swiggy/UberEats/DoorDash) into POS+KDS.
- Accounting export (QuickBooks/Tally); webhooks API.

11 Admin / back-office MVP

- Org settings, outlets, taxes, receipt templates, hardware (printers/KDS) config.
- Staff & roles, audit log, subscription & billing, data import/export.

5 · Key screens to build

POS terminal

- Order entry + cart
- Tables / floor map
- Payment & receipt
- Shift / cash drawer

KDS

- Station ticket board
- Timers & bump
- All-day view

Manager web app

- Dashboard
- Menu builder
- Inventory & POs
- Reports
- Staff & roles

Diner (QR)

- Menu & cart
- Order status
- Split & pay

6 · Core data model (starter schema)

Table	Key fields
organizations	id, name, plan, trial_ends_at, status
outlets	id, org_id, name, address, timezone, tax_config
users / staff_roles	id, org_id, outlet_id, role, email
categories / menu_items	id, outlet_id, name, price, tax_class, station_id, is_available
modifiers / item_modifiers	id, name, price_delta, group, required
tables	id, outlet_id, label, seats, status, qr_token
orders	id, outlet_id, type, table_id, status, subtotal, tax, total, customer_id
order_items	id, order_id, menu_item_id, qty, modifiers, station_id, kds_status
payments	id, order_id, method, amount, provider_ref, status
ingredients	id, outlet_id, name, unit, stock_qty, min_qty
recipes / recipe_items	menu_item_id, ingredient_id, quantity
purchase_orders	id, supplier_id, status, items, total
suppliers	id, outlet_id, name, contact
customers	id, org_id, name, phone, loyalty_points, total_spend
shifts / cash_movements	id, outlet_id, user_id, opened_at, closed_at, opening_cash, closing_cash
audit_log	id, org_id, user_id, entity, action, before, after, created_at

RLS: every table carries org_id (and outlet_id where relevant). Policies: a user can only read/write rows for their organization and assigned outlets, gated by role.

7 · API surface (REST/RPC, examples)

- POST /auth/signup · POST /outlets · POST /staff/invite
- GET/POST /menu/items · PATCH /menu/items/:id
- POST /orders · PATCH /orders/:id · POST /orders/:id/pay
- GET /kds/tickets(realtime) · PATCH /kds/items/:id
- GET/POST /inventory/ingredients · POST /inventory/purchase-orders
- GET /reports/sales?from&to&outlet · POST /webhooks/aggregator

8 · Non-functional requirements

Reliability • Offline-first POS w/ sync queue • 99.9% uptime target • Daily backups	Security • RLS + RBAC enforced • Audit log on money/stock • PCI-safe payments (provider-hosted) • HTTPS, rate limiting
Performance • POS actions < 200ms • Realtime KDS < 1s • Indexed queries, pagination	Scale • 1 → 1000 outlets per org • Tenant-isolated data • Horizontal API scaling

9 · Build roadmap (phase order)

Phase	Ship
MVP	Auth + multi-tenant + RBAC, Menu, POS & Billing (offline) , KDS, basic admin. Goal: take orders & get paid.
V1	Inventory & recipe costing, QR/online ordering, multi-outlet, analytics, payments & aggregators.
V2	CRM & loyalty, campaigns, advanced reports, accounting integrations, mobile apps.

10 · Acceptance criteria (definition of done)

- A new owner can sign up, create an outlet, build a menu, and take a paid order in under 10 minutes.
- POS continues billing with the network off and reconciles correctly on reconnect.
- Settling a bill auto-deducts recipe stock; low stock triggers an alert/PO.
- KDS updates in realtime across devices; ready items notify the waiter.
- All data is tenant-isolated (verified by RLS tests); money/stock changes are audited.
- Reports match POS totals to the cent for any date range/outlet.